

MATH 231 – Linear Algebra I

The Computational Unit, Part 1 ■ Counting Operations

Kiely Hall 326 ■ Instructor: Kevin Bedoya

About this document. This opens a new unit, on the computational side of the subject: how much work a calculation actually costs, how numbers are really stored, and how the algorithms of this course are built and analysed.

A common confusion (Section numbers restart). This unit numbers its sections from 1 again — they do *not* continue from the vector unit. So a reference to “§1.3” below means §1.3 *of this unit*. Whenever we point back at the earlier material we will say so explicitly, as in “the vector unit, §11.3”. Both units have a §11.3, and they are different sections.

Assumed from the vector unit. Vectors in $\mathbb{R}^n$ and componentwise addition (§7 there); the dot product $\langle \mathbf{u}, \mathbf{v} \rangle = \sum_i u_i v_i$ (§11.3); the Euclidean norm (§2); the projection $\text{proj}_{\mathbf{u}}(\mathbf{v})$ (§12.4); the identity $\langle \mathbf{u}, \mathbf{u} \rangle = \|\mathbf{u}\|^2$ (§11.6); orthogonality and cosine similarity (§11.4, §11.7); and linear dependence (§18.4).

What you should be able to do afterwards. Say what an *operation* is; count exactly how many operations vector addition, the dot product, the norm and the projection require in $\mathbb{R}^n$; recognise redundant work in a formula and remove it; and explain why an exact count is worth having.

1. Counting operations

1.1. A first count: adding two vectors

Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ and set $\mathbf{w} = \mathbf{u} + \mathbf{v}$. Here is a question we have never asked before:

How many calculations must a machine perform in order to produce $\mathbf{w}$?

Before answering, one condition. Addition of vectors — like every vector operation in this course — is defined only when both inputs have the *same* dimension. There is no sum of a vector in $\mathbb{R}^3$ and a vector in $\mathbb{R}^5$: the definition matches up first coordinate with first, second with second, and would run out. So we may take both to have exactly n coordinates.

Now count. Addition is componentwise, so

$$\mathbf{w} = [u_1 + v_1, u_2 + v_2, \dots, u_n + v_n].$$

Each slot of the answer requires exactly one addition of two real numbers, and there are n slots. Nothing else happens. So producing $\mathbf{w}$ costs

n operations

and the same count holds for $\mathbf{u} - \mathbf{v}$, which differs only in replacing each addition by a subtraction.

1.2. What counts as an operation

Definition 1.1 (Operation). In a computational setting, an **operation** is a single calculation carried out on scalar quantities by one operator — one addition, one subtraction, one multiplication, one division, one square root.

Two pieces of vocabulary we will use constantly: a scalar addition is an **add**, and a scalar multiplication is a **mult**. So the count above reads: vector addition in $\mathbb{R}^n$ costs n adds.

Remark. Counting every operation as costing “one” is a simplification, and a deliberate one. On real hardware a multiplication is somewhat slower than an addition, a division is several times slower again, and a square root slower still. We ignore that for now because the first thing worth knowing about a computation is *how the count grows with n* , and that pattern is the same whichever weights you attach. We will revisit the assumption when it starts to matter.

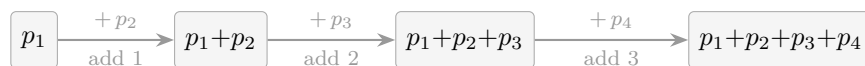
1.3. The dot product

Now take $\langle \mathbf{u}, \mathbf{v} \rangle$. Write out what the definition asks for:

$$\langle \mathbf{u}, \mathbf{v} \rangle = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n.$$

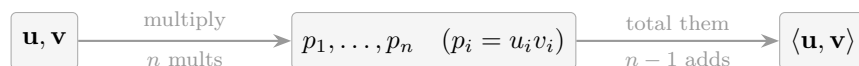
Two distinct stages are visible. First, n products $u_i v_i$ must be formed — that is n mults. Then those n numbers must be added into a single total.

How many adds does it take to total n numbers? Not n . Adding is a *binary* operation, so each add consumes two numbers and returns one, reducing the count of outstanding numbers by exactly one. Starting with n numbers and finishing with 1 therefore takes $n - 1$ adds.



4 numbers, 3 adds — in general n numbers, $n - 1$ adds

So the dot product runs in two stages:



Adding the two stages,

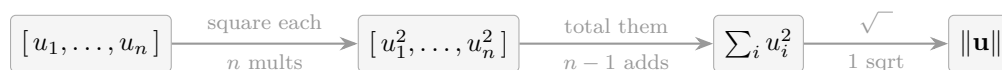
$$\langle \mathbf{u}, \mathbf{v} \rangle \text{ costs } n + (n - 1) = 2n - 1 \text{ operations}$$

1.4. The norm

Next, $\|\mathbf{u}\|$. Written out term by term,

$$\|\mathbf{u}\| = (u_1^2 + u_2^2 + \cdots + u_n^2)^{1/2}.$$

Now three stages. Each u_i^2 is the product $u_i \cdot u_i$, so squaring all n coordinates is n mults. Totalling those n squares is $n - 1$ adds, by the counting argument of §1.3. And finally one square root is taken, of a single scalar — one operation, no matter how large n is, because by that point only one number is left.



Adding the three stages,

$$\|\mathbf{u}\| \text{ costs } n + (n - 1) + 1 = 2n \text{ operations}$$

Remark. It is worth noticing where the 1 sits. The square root is applied *once*, at the very end, to a scalar — not once per coordinate. Miscounting this is the most common error in a first operation count: the stages of a computation operate on different numbers of quantities, and you must track how many are still in play at each stage.

1.5. The projection

Now something with several moving parts. Recall from the vector unit, §12.4,

$$\text{proj}_{\mathbf{u}}(\mathbf{v}) = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\|} \cdot \frac{\mathbf{u}}{\|\mathbf{u}\|}.$$

The hard work is already done: we know what a dot product and a norm cost. What remains is to take the formula apart and add up the pieces, being careful at each step about whether the quantity involved is a scalar or a vector.

1. $\langle \mathbf{u}, \mathbf{v} \rangle$ — a dot product, so $2n - 1$ operations.
2. $\|\mathbf{u}\|$ — a norm, so $2n$ operations. The formula asks for it *twice*, once in each factor, so as written that is $4n$.
3. $\langle \mathbf{u}, \mathbf{v} \rangle / \|\mathbf{u}\|$ — careful here. Both of these are *scalars*, so this is a single scalar division: 1 operation.
4. $\mathbf{u} / \|\mathbf{u}\|$ — here $\mathbf{u}$ is a *vector*, and dividing it by a scalar is componentwise. That is n divisions, so n operations.
5. Finally the scalar from step 3 multiplies the vector from step 4, again componentwise: n mults, so n operations.

Adding them up,

$$(2n - 1) + 4n + 1 + n + n = 8n,$$

so the formula *as written* costs $8n$ operations.

1.6. Doing it better

Is there anything wrong with that count? Mathematically, no — $8n$ is the honest cost of evaluating the expression exactly as it appears. But to anyone who has written code, the problem is glaring.

First saving: stop computing the same thing twice

Why did we compute $\|\mathbf{u}\|$ *twice*? The two occurrences are the same number. Compute it once, keep it, and use it in both places. That deletes one of the two $2n$ charges:

$$(2n - 1) + 2n + 1 + n + n = 6n.$$

Nothing about the mathematics changed — only the bookkeeping. This is the single most common optimisation there is, and it has a name in programming: *common subexpression elimination*.

Second saving: do not take a square root you are going to square

Look at what $\|\mathbf{u}\|$ is used for. It appears twice, and both times it divides — so what the formula really uses is $\|\mathbf{u}\|^2$. And by the vector unit's §11.6,

$$\|\mathbf{u}\|^2 = \langle \mathbf{u}, \mathbf{u} \rangle,$$

which is a dot product: $2n - 1$ operations, with *no square root at all*. Rewriting the projection in the equivalent form already given in §12.4,

$$\text{proj}_{\mathbf{u}}(\mathbf{v}) = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\|^2} \mathbf{u},$$

the count becomes

$$\underbrace{(2n-1)}_{\langle \mathbf{u}, \mathbf{v} \rangle} + \underbrace{(2n-1)}_{\langle \mathbf{u}, \mathbf{u} \rangle} + \underbrace{1}_{\text{one scalar division}} + \underbrace{n}_{\text{scale } \mathbf{u}} = 5n - 1.$$

Notice the third saving hiding in that line. The old version made two separate passes over $\mathbf{u}$ — dividing by the norm, then multiplying by the scalar — costing $n + n$. Combining the two scalars into one *before* touching the vector reduces that to a single pass of n mults.

Version	Cost	What changed
As written	$8n$	—
Reuse $\ \mathbf{u}\ $	$6n$	compute the norm once, not twice
Use $\ \mathbf{u}\ ^2 = \langle \mathbf{u}, \mathbf{u} \rangle$	$5n - 1$	no square root; one pass over $\mathbf{u}$

For $n = 1000$ that is 8000 operations against 4999 — the same answer, at a little over half the cost. And the square root, which we noted in §1.2 is among the slowest operations available, has been removed entirely.

1.7. Why count at all

A fair question at this point is why any of this matters. The formulas are correct either way, and a modern machine will evaluate all three versions faster than you can notice.

The reason is that we are stepping out of mathematics and into computation. Until now every object in this course has been exact and every operation free: $\mathbb{R}^n$ is infinite, a real number carries infinitely many digits, and writing $\|\mathbf{u}\|$ costs nothing because writing is not computing. A machine has none of that. It has finitely many digits per number, finitely many operations per second, and finite memory, and every one of those limits will eventually bind.

The computations here are small, and modern hardware and libraries are heavily optimised for exactly them. But the habit is what matters: taking a formula apart, asking what each stage costs, and noticing when the same quantity is being computed twice. Do that on a projection and you save a square root. Do it on an algorithm that runs a projection a million times inside a loop, and the difference is the difference between a program that finishes and one that does not.

Looking ahead. Two threads start here. First, we have been counting *exactly*, producing answers like $5n - 1$; usually the constants and the -1 do not matter and only the growth rate does, which is what *asymptotic* notation captures — the reason $5n - 1$ and $8n$ are “the same” in a sense worth making precise. Second, we have assumed a machine stores real numbers. It does not; it stores *floating-point* numbers, of which there are finitely many, and arithmetic on them is not quite the arithmetic of $\mathbb{R}$. Both are next.

1.8. Exercises

In each case, count the operations required, and keep the count *as low as possible* — part of the exercise is noticing what the given information lets you skip.

- E1.** Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{200}$ be orthonormal. Count the operations needed to compute $\text{proj}_{\mathbf{u}}(\mathbf{v})$.
- E2.** Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{50}$ with $\cos(\mathbf{u}, \mathbf{v}) = 1$. Count the operations needed to compute $\|\mathbf{u} - \mathbf{v}\|$.
- E3.** Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{25}$ be nonzero and linearly dependent. Count the operations needed to compute $\cos(\mathbf{u}, \mathbf{v})$.
- E4.** Let $\mathbf{u}, \mathbf{v}, \mathbf{w} \in \mathbb{R}^{100}$. Count the operations needed to compute $\langle \mathbf{u}, \mathbf{v} \rangle + \mathbf{w}$.

Answer key

E1. *Zero operations.* “Orthonormal” means both vectors are unit length *and* orthogonal, so $\langle \mathbf{u}, \mathbf{v} \rangle = 0$ is given. Then

$$\text{proj}_{\mathbf{u}}(\mathbf{v}) = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\|^2} \mathbf{u} = \frac{0}{1} \mathbf{u} = \mathbf{0},$$

and the answer is the zero vector, written down without arithmetic. Ignoring the hypothesis and evaluating the optimised formula would have cost $5n - 1 = 999$ operations to arrive at the same $\mathbf{0}$. The lesson is that recognising structure beats computing.

E2. $3n = 150$ operations, and the hypothesis does not help. Directly: $\mathbf{u} - \mathbf{v}$ costs $n = 50$ subtractions, and taking the norm of the result costs $2n = 100$, for $3n = 150$ in total.

It is worth seeing why the given information is no use. Since $\cos(\mathbf{u}, \mathbf{v}) = 1$ the two vectors point the same way, and expanding as in the vector unit’s §13 gives $\|\mathbf{u} - \mathbf{v}\|^2 = \|\mathbf{u}\|^2 - 2\langle \mathbf{u}, \mathbf{v} \rangle + \|\mathbf{v}\|^2 = (\|\mathbf{u}\| - \|\mathbf{v}\|)^2$, so $\|\mathbf{u} - \mathbf{v}\| = \|\mathbf{u}\| - \|\mathbf{v}\|$. That is a genuine shortcut in appearance, but it costs two norms and a subtraction: $2n + 2n + 1 = 4n + 1 = 201$, which is *more* than 150. A shortcut is only a shortcut if it is cheaper.

E3. *1 operation.* Linearly dependent and nonzero means parallel (vector unit, §18.4), so $\mathbf{v} = \alpha \mathbf{u}$ for some $\alpha \neq 0$, and the angle between them is 0 or π . Hence $\cos(\mathbf{u}, \mathbf{v}) = \pm 1$, with the sign being the sign of α . To find that sign, pick any index i with $u_i \neq 0$ and form $\alpha = v_i/u_i$: one division. Then $\cos(\mathbf{u}, \mathbf{v}) = +1$ if $\alpha > 0$ and -1 if $\alpha < 0$.

Evaluating the definition instead would cost one dot product, two norms, one mult and one division: $(2n - 1) + 2n + 2n + 1 + 1 = 6n + 1 = 151$.

E4. *The expression is undefined, so there is nothing to count.* By the signatures of the vector unit, $\langle \cdot, \cdot \rangle$ maps $\mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$, so $\langle \mathbf{u}, \mathbf{v} \rangle$ is a *scalar*; and $+$ maps $\mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}^n$, so it requires two *vectors*. A scalar plus a vector is not defined — exactly the same-dimension requirement noted in §1.1, in its most extreme form.

Remark (Why E4 is not a trick question). Type mismatches of this kind are the most common bug in numerical code, and they do not always announce themselves. Written in NumPy or MATLAB, the expression $\mathbf{u}@\mathbf{v} + \mathbf{w}$ will *not* raise an error: those languages *broadcast* the scalar, quietly adding it to every entry of $\mathbf{w}$ at a cost of $n = 100$ adds. That may be what you wanted, or it may be a bug that now produces plausible wrong numbers instead of a crash. The mathematics and the software disagree here, and knowing which one you are speaking is part of the job.